AirPen Glove

Laptop Edition — No Raspberry Pi, No OLED

GY-BNO055 + 2 Flex Sensors + ESP32 + Your Laptop

Simpler setup — everything runs on your laptop

- ✓ ESP32 connects to laptop via **USB cable** (no Bluetooth pairing)
- ✓ Output shown on **laptop screen** (no OLED wiring)
- ✓ Recognition runs on **laptop CPU** (much faster than Pi)
- ✓ No Raspberry Pi setup, no SD card, no Linux config
- ✓ Same `airwrite.py` you already have — only 3 lines change

You already have:	BNO055, 2x flex sensors, ESP32, glove, breadboard, wires, double-sided tape
Still need to buy:	10kΩ resistors (10 pcs, ≈20 BDT)
Build time:	2 hours
Skill level:	Complete beginner
Project:	AirPen — Computer Interfacing, MIST

Contents

1. What Changes Without Pi and OLED

Table 1: Laptop edition vs Raspberry Pi edition

Part	Pi edition	Laptop edition (this guide)
Data link	Bluetooth pairing	**USB cable — plug and play**
Processing	Raspberry Pi 4 CPU	**Your laptop CPU (10x faster)**
Output display	128×64 OLED screen	**Laptop screen (OpenCV window)**
Power for glove	LiPo battery + charger	**USB from laptop**
Setup time	3–4 hours	**30 minutes**
Extra parts needed	Pi, SD card, OLED, battery	**Nothing extra**

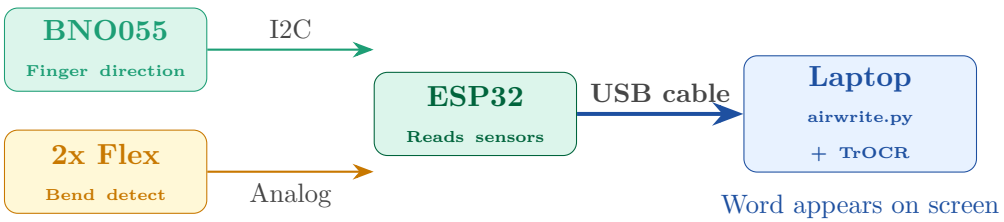
Removing the Pi and OLED eliminates the two hardest parts of the build — Linux configuration and Bluetooth pairing. The USB connection is instant and never drops. Your laptop runs TrOCR or Google Cloud Vision far faster than a Pi ever could, so recognition takes under 1 second instead of 4–6 seconds.

2. One Thing To Buy

You need **10kΩ resistors** — buy a pack of 10 from any electronics shop. Without them the flex sensors output meaningless values.

How to identify: colour bands are **Brown – Black – Orange – Gold**.

3. How The System Works



3.1. What Each Part Does

GY-BNO055 — Mounted on your index finger. Contains an accelerometer, gyroscope and magnetometer, and fuses all three internally to give clean orientation angles. As you move your finger to write, this reports which direction it is pointing 100 times per second.

Flex sensors — Strips that change resistance when bent. One on the index finger detects "pen down". One on the middle finger triggers word recognition. They replace the thumb-touch and peace-sign gestures from the camera version.

ESP32 over USB — Reads all sensors and streams the data as CSV lines over the USB serial port. Your laptop reads this exactly like reading from any serial device. No Bluetooth, no pairing, no drivers to install on most systems.

4. Complete Circuit Wiring

4.1. Full Pin Connection Table

Table 2: Every wire connection — follow this exactly

Wire	From	Pin	To ESP32	Notes
Red	BNO055	VIN	3.3V	NEVER 5V
Black	BNO055	GND	GND	Any GND pin
Blue	BNO055	SDA	GPIO 21	I2C data line
Yellow	BNO055	SCL	GPIO 22	I2C clock line
Red	Flex 1 (index)	Pin 1	3.3V	Top of divider
Purple	Flex 1 (index)	Pin 2	GPIO 34	Midpoint read
Black	10kΩ #1	Leg 2	GND	Bottom of divider
Red	Flex 2 (middle)	Pin 1	3.3V	Top of divider
Purple	Flex 2 (middle)	Pin 2	GPIO 35	Midpoint read
Black	10kΩ #2	Leg 2	GND	Bottom of divider
—	ESP32	USB	**Laptop USB port**	Power + data + programming

The BNO055 runs on **3.3V only**. Connecting VIN to the ESP32's 5V pin will permanently kill the chip within seconds. Trace the red wire with your finger and confirm it goes to the pin labelled **3V3** or **3.3V**, not **VIN** or **5V** on the ESP32.

4.2. Wiring Diagram

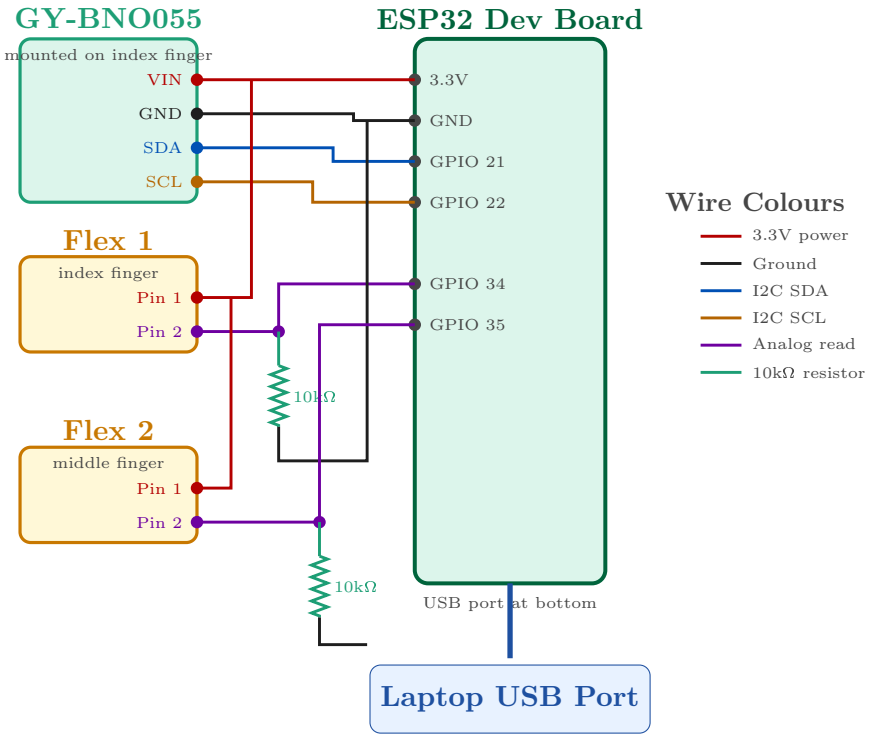


Figure 1: Complete wiring — BNO055 and two flex sensors to ESP32, ESP32 to laptop via USB

4.3. Breadboard Placement

Table 3: Where each component goes on your breadboard

Row	Column	Component / connection
Power rails (long strips along the edges)		
—	red (+) rail	Wire from ESP32 3.3V pin
—	blue (-) rail	Wire from ESP32 GND pin
ESP32 board		
1–15	a–e, f–j	ESP32 straddling the centre gap
BNO055 sensor		
20	a	BNO055 VIN wire
20	b	Jumper to red (+) rail
21	a	BNO055 GND wire
21	b	Jumper to blue (-) rail
22	a	BNO055 SDA wire
22	b	Jumper to ESP32 GPIO 21
23	a	BNO055 SCL wire
23	b	Jumper to ESP32 GPIO 22
Flex sensor 1 (index finger)		
26	a	Flex 1 Pin 1
26	b	Jumper to red (+) rail
27	a	Flex 1 Pin 2
27	b	Jumper to ESP32 GPIO 34
27	c	10kΩ resistor leg 1
28	c	10kΩ resistor leg 2
28	d	Jumper to blue (-) rail
Flex sensor 2 (middle finger)		
31	a	Flex 2 Pin 1
31	b	Jumper to red (+) rail
32	a	Flex 2 Pin 2
32	b	Jumper to ESP32 GPIO 35
32	c	10kΩ resistor leg 1
33	c	10kΩ resistor leg 2
33	d	Jumper to blue (-) rail

Each numbered row has 5 holes on the left (a–e) and 5 on the right (f–j). All 5 holes in a row on the same side are electrically connected. So if you put a wire in 27a and another in 27b, they are connected. The gap in the middle separates left from right — that is where the ESP32 straddles.

5. Physical Glove Assembly

Put the glove on your writing hand and mark with a pen:

- **Zone A** — top of index finger, middle segment (BNO055 goes here)
- **Zone B** — along index finger, base to tip (flex sensor 1)
- **Zone C** — along middle finger, base to tip (flex sensor 2)
- **Zone D** — flat area on back of hand (breadboard)

Take the glove off. These marks guide every mounting step.

1. Cut double-sided tape to the size of the BNO055 board
2. Stick it to the underside of the module (the side without the chip)
3. Press onto Zone A firmly, hold 45 seconds
4. **Check alignment:** view the finger from the side — the board must be *parallel* to the finger, not tilted

The BNO055 reports orientation relative to its own body. If the board is tilted 15 degrees, every reading is offset by 15 degrees, and your written letters will come out skewed. Fix the alignment now — it cannot be corrected in software.

1. Lay flex sensor 1 along Zone B (top of index finger, base to tip)
2. Secure with 3 small pieces of tape — base, middle, near tip
3. Repeat with flex sensor 2 along Zone C on the middle finger
4. **Test:** bend each finger — the sensor should flex smoothly, not buckle or crease

1. Apply double-sided tape to the bottom of the breadboard
2. Press onto Zone D
3. Seat the ESP32 on the breadboard, straddling the centre gap
4. Orient the ESP32 so its **USB port faces the wrist** — the cable then runs down your arm instead of across your fingers

1. Run wires along the **side** of each finger, never across the palm
2. Tape every 2cm with small strips
3. **Leave a 2cm loop of slack at every knuckle** — this is the single most important detail
4. Connect each wire end to the breadboard per the table in Section 4.3

Forgetting slack at the knuckles. When you bend a finger, the distance the wire must cover increases. Without slack the wire pulls on the solder joint, causing intermittent dropouts that look like random sensor failures and take hours to diagnose.

6. ESP32 Firmware

6.1. Arduino IDE Setup

1. Download Arduino IDE from `arduino.cc/en/software`
2. **File** → **Preferences** → Additional boards manager URLs, paste:
   `https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json`
3. **Tools** → **Board** → **Boards Manager** → search "esp32" → Install
4. **Tools** → **Board** → select **ESP32 Dev Module**
5. **Tools** → **Manage Libraries** → install **Adafruit BNO055** and **Adafruit Unified Sensor**
6. Plug the ESP32 into your laptop, then **Tools** → **Port** → select the COM port that appears

Install the CP2102 or CH340 USB driver — which one depends on your ESP32 board. Search "CP2102 driver" or "CH340 driver", install, then unplug and replug the ESP32.

6.2. Complete Firmware — USB Serial Version

Listing 1: glove_firmware.ino - upload this to the ESP32

```
// ============================================================
// AirPen Glove Firmware - LAPTOP / USB EDITION
// Reads BNO055 orientation + 2 flex sensors
// Streams CSV over USB serial at 100 Hz
// ============================================================

#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BNO055.h>

//        Pin definitions (match the wiring table)
```

```
#define FLEX_INDEX_PIN   34   // flex sensor on index finger
#define FLEX_MIDDLE_PIN  35   // flex sensor on middle finger
#define SDA_PIN          21   // I2C data
#define SCL_PIN          22   // I2C clock

//        BNO055 object
Adafruit_BNO055 bno = Adafruit_BNO055(55, 0x28);

//        Flex thresholds - CALIBRATE THESE (see Section 6.3)
int FLEX_INDEX_THRESHOLD  = 2700;  // above this = pen DOWN
int FLEX_MIDDLE_THRESHOLD = 2700;  // above this = recognise gesture

//        Send rate
unsigned long lastSend = 0;
const int RATE_MS = 10;            // 10ms = 100 samples/second

void setup() {
  // USB serial to laptop - must match the Python side
  Serial.begin(115200);
  delay(500);

  Serial.println("# AirPen Glove starting");

  // Start I2C on the pins we wired
  Wire.begin(SDA_PIN, SCL_PIN);

  if (!bno.begin()) {
    Serial.println("# ERROR: BNO055 not found");
    Serial.println("# Check VIN->3.3V, GND->GND, SDA->21, SCL->22");
    while (1) delay(1000);
  }

  delay(500);
  bno.setMode(OPERATION_MODE_NDOF);   // full 9-DOF fusion
  delay(1000);

  Serial.println("# BNO055 OK - calibrating");
  Serial.println("# Move the glove in a figure-8 for ~20 seconds");

  // Wait until the system calibration is at least 2 of 3
  uint8_t sys = 0, gyro = 0, accel = 0, mag = 0;
  int tries = 0;
  do {
    bno.getCalibration(&sys, &gyro, &accel, &mag);
    // Lines starting with # are status messages, not data
    Serial.printf("# CAL sys=%d gyro=%d accel=%d mag=%d\n",
                  sys, gyro, accel, mag);
    delay(500);
    tries++;
  } while (sys < 2 && tries < 60);

  Serial.println("# READY");
}

void loop() {
  unsigned long now = millis();
  if (now - lastSend < RATE_MS) return;
  lastSend = now;

  //        Orientation from BNO055 (degrees)
  // x = heading  (0-360)   --> maps to canvas X
  // y = pitch    (-90..90) --> maps to canvas Y
  // z = roll     (-180..180) --> unused for drawing
  imu::Vector<3> e = bno.getVector(Adafruit_BNO055::VECTOR_EULER);

  //        Flex sensors (0-4095, higher = more bent)
  int flexIndex  = analogRead(FLEX_INDEX_PIN);
  int flexMiddle = analogRead(FLEX_MIDDLE_PIN);

  bool penDown = (flexIndex  > FLEX_INDEX_THRESHOLD);
  bool gesture = (flexMiddle > FLEX_MIDDLE_THRESHOLD);

  //        One CSV line per sample
```

```
  // heading,pitch,roll,flexIndex,flexMiddle,penDown,gesture
  Serial.print(e.x(), 2);       Serial.print(',');
  Serial.print(e.y(), 2);       Serial.print(',');
  Serial.print(e.z(), 2);       Serial.print(',');
  Serial.print(flexIndex);      Serial.print(',');
  Serial.print(flexMiddle);     Serial.print(',');
  Serial.print(penDown ? 1 : 0); Serial.print(',');
  Serial.println(gesture ? 1 : 0);
}
```

6.3. Calibrating the Flex Sensors

1. Upload the firmware, then open **Tools → Serial Monitor** at **115200 baud**
2. Hold your index finger completely straight — note the 4th number in each line
3. Bend the index finger to a comfortable "closed" position — note the 4th number again
4. Set `FLEX_INDEX_THRESHOLD` to roughly halfway between those two values
5. Repeat with the middle finger for the 5th number, set `FLEX_MIDDLE_THRESHOLD`
6. Re-upload the firmware with your values

Straight index reads **1850**. Bent index reads **3450**. Halfway is $(1850 + 3450)/2 = 2650$. Set `FLEX_INDEX_THRESHOLD = 2650`.

7. Laptop Software

7.1. Install Python Package

Only one package is needed beyond what you already have:

```
pip install pyserial
```

7.2. Finding Your ESP32 Serial Port

OS	Port looks like	How to find it
Windows	`COM3`, `COM5`	Device Manager → Ports (COM & LPT)
Linux	`/dev/ttyUSB0`	Run `ls /dev/ttyUSB*`
macOS	`/dev/cu.usbserial-*`	Run `ls /dev/cu.*`

Or list them from Python:

```
python -c "import serial.tools.list_ports as p; [print(x.device,
   '-', x.description) for x in p.comports()]"
```

7.3. glove_reader.py — Save This Next To airwrite.py

Listing 2: glove_reader.py - reads the glove over USB serial

```
# ============================================================
# AirPen Glove Reader - LAPTOP / USB EDITION
# Reads CSV from ESP32 over USB serial in a background thread
# Converts BNO055 orientation into canvas X-Y coordinates
# ============================================================

import serial
import threading
import time

#        CONFIGURE THESE
SERIAL_PORT = "COM3"        # Windows: COM3 | Linux: /dev/ttyUSB0
BAUD_RATE   = 115200
CANVAS_W    = 1280          # must match your airwrite.py canvas
CANVAS_H    = 720

# How many canvas pixels per degree of finger rotation.
# Bigger  = letters get bigger with less hand movement.
# Smaller = you must move your hand more to draw the same size.
SCALE = 12.0

# Ignore movements smaller than this (degrees) - kills sensor noise
DEADZONE = 0.35

#        Shared state, guarded by a lock
_state = {
    "canvas_x": CANVAS_W // 2,
    "canvas_y": CANVAS_H // 2,
    "pen_down": False,
    "gesture":  False,
    "ready":    False,
    "flex_i":   0,
    "flex_m":   0,
}
_lock = threading.Lock()

#        Position integration state
_pos_x = float(CANVAS_W // 2)
_pos_y = float(CANVAS_H // 2)
_prev_heading = None
_prev_pitch   = None
_running      = False


def _parse(line):
    """Parse one CSV line. Lines starting with # are status messages."""
    line = line.strip()
    if not line or line.startswith("#"):
        return None
    parts = line.split(",")
    if len(parts) != 7:
        return None
    try:
        return {
            "heading":  float(parts[0]),
            "pitch":    float(parts[1]),
            "roll":     float(parts[2]),
            "flex_i":   int(parts[3]),
            "flex_m":   int(parts[4]),
            "pen_down": parts[5] == "1",
            "gesture":  parts[6] == "1",
        }
    except ValueError:
        return None


def _integrate(heading, pitch):
    """Turn change-in-angle into change-in-position on the canvas."""
    global _pos_x, _pos_y, _prev_heading, _prev_pitch
```

```
    if _prev_heading is None:
        _prev_heading, _prev_pitch = heading, pitch
        return int(_pos_x), int(_pos_y)

    d_h = heading - _prev_heading
    d_p = pitch   - _prev_pitch

    # Heading wraps at 360 - fix the jump
    if d_h >  180: d_h -= 360
    if d_h < -180: d_h += 360

    # Dead zone kills jitter when holding still
    if abs(d_h) < DEADZONE: d_h = 0.0
    if abs(d_p) < DEADZONE: d_p = 0.0

    _pos_x += d_h * SCALE
    _pos_y -= d_p * SCALE       # minus so tilting up moves up on screen

    _pos_x = max(0.0, min(float(CANVAS_W - 1), _pos_x))
    _pos_y = max(0.0, min(float(CANVAS_H - 1), _pos_y))

    _prev_heading, _prev_pitch = heading, pitch
    return int(_pos_x), int(_pos_y)


def _reader(port, baud):
    """Background thread - never blocks the main airwrite loop."""
    global _running
    print(f"[glove] opening {port} @ {baud}...")
    try:
        ser = serial.Serial(port, baud, timeout=1.0)
    except serial.SerialException as ex:
        print(f"[glove] CANNOT OPEN {port}: {ex}")
        print("[glove] Close the Arduino Serial Monitor and try again.")
        return

    time.sleep(2.0)          # ESP32 resets when the port opens
    ser.reset_input_buffer()
    print("[glove] connected - waiting for calibration")
    _running = True

    while _running:
        try:
            raw = ser.readline().decode("utf-8", errors="ignore")

            # Print status lines so you can see calibration progress
            if raw.startswith("#"):
                print("[glove]", raw.strip())
                if "READY" in raw:
                    with _lock:
                        _state["ready"] = True
                continue

            d = _parse(raw)
            if d is None:
                continue

            cx, cy = _integrate(d["heading"], d["pitch"])
            with _lock:
                _state["canvas_x"] = cx
                _state["canvas_y"] = cy
                _state["pen_down"] = d["pen_down"]
                _state["gesture"]  = d["gesture"]
                _state["flex_i"]   = d["flex_i"]
                _state["flex_m"]   = d["flex_m"]
                _state["ready"]    = True

        except Exception as ex:
            print(f"[glove] read error: {ex}")
            time.sleep(0.1)

    ser.close()
```

```

def start(port=SERIAL_PORT, baud=BAUD_RATE):
    """Call once at startup, before the main loop."""
    t = threading.Thread(target=_reader, args=(port, baud), daemon=True)
    t.start()


def stop():
    global _running
    _running = False


def get_state():
    """
    Call every frame from airwrite.py.
    Returns (x, y, pen_down, gesture, ready)
    """
    with _lock:
        return (_state["canvas_x"], _state["canvas_y"],
                _state["pen_down"], _state["gesture"],
                _state["ready"])


def get_flex():
    """Raw flex values - useful for on-screen debugging."""
    with _lock:
        return _state["flex_i"], _state["flex_m"]


def recenter():
    """Reset the cursor to canvas centre - call when a new word starts."""
    global _pos_x, _pos_y, _prev_heading, _prev_pitch
    _pos_x = float(CANVAS_W // 2)
    _pos_y = float(CANVAS_H // 2)
    _prev_heading = None
    _prev_pitch   = None
```

8. Integrating With airwrite.py

Three edits. Nothing else in your file changes — the canvas, TrOCR or Cloud Vision, word mode, and all the drawing code stay exactly as they are.

8.1. Edit 1 — Import and start the reader

Add near the top of `airwrite.py`, after your other imports:

```
import glove_reader

# Start the background USB reader. Change the port to match your machine.
glove_reader.start(port="COM3")      # Linux: "/dev/ttyUSB0"
```

8.2. Edit 2 — Replace the camera input block

Find the block in your main loop that reads MediaPipe landmarks:

```
# ---- OLD (camera version) ----
if lm:
    draw_hand(frame, lm, w, h)
    tip_x, tip_y = get_smooth_tip(lm, w, h)
    gesture      = detect_gesture(lm)
    pen_down     = get_pen_down(lm)
```

Replace it with:

```
# ---- NEW (glove version) ----
tip_x, tip_y, pen_down, glove_gesture, glove_ready = glove_reader.get_state()

if not glove_ready:
    cv2.putText(frame, "Waiting for glove... move it in a figure-8",
                (20, 90), cv2.FONT_HERSHEY_SIMPLEX, 0.7,
                (0, 220, 255), 2)
    gesture = "none"
else:
    # Bending the middle finger acts as the "recognise" trigger,
    # exactly where the peace sign used to be.
    gesture = "peace" if glove_gesture else "index_up"

    # Draw the cursor so you can see where the pen is on the canvas
    cv2.circle(canvas, (tip_x, tip_y),
               8 if pen_down else 5,
               (0, 60, 200) if pen_down else (60, 60, 60), -1)
```

8.3. Edit 3 — Recentre when a new word starts

Find every place where the state becomes `"drawing"` and add one line after it:

```
state = "drawing"
glove_reader.recenter()      # <-- add this line
```

8.4. Optional — Show flex values on screen while tuning

Handy while you are dialling in the thresholds:

```
fi, fm = glove_reader.get_flex()
cv2.putText(frame, f"Flex index: {fi}   middle: {fm}",
            (20, h - 130), cv2.FONT_HERSHEY_SIMPLEX,
            0.5, (180, 180, 180), 1)
```

8.5. Control Mapping

Table 4: Camera gestures vs glove actions

Camera version	Glove version	Result
Move index finger	Move gloved hand	Cursor moves on canvas
Thumb touches index	**Bend index finger**	Pen DOWN — stroke drawn
Lift thumb	**Straighten index**	Pen UP — move without drawing
Peace sign	**Bend middle finger**	Recognise the word
Open palm	Press `C` on keyboard	Clear the canvas
—	Press `U` on keyboard	Undo the last stroke

9. Testing — Run These In Order

Upload this alone first:

```
#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BNO055.h>
Adafruit_BNO055 bno = Adafruit_BNO055(55, 0x28);
void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);
  Serial.println(bno.begin() ? "BNO055 OK" : "BNO055 NOT FOUND");
}
void loop() {
  imu::Vector<3> e = bno.getVector(Adafruit_BNO055::VECTOR_EULER);
  Serial.printf("H %.1f  P %.1f  R %.1f\n", e.x(), e.y(), e.z());
  delay(200);
}
```

Pass: "BNO055 OK" and three numbers that change as you tilt the sensor.
Fail: confirm VIN is on **3.3V** not 5V, SDA on GPIO21, SCL on GPIO22.

```
void setup() { Serial.begin(115200); }
void loop() {
  Serial.printf("index %d   middle %d\n",
                analogRead(34), analogRead(35));
  delay(120);
}
```

Pass: both numbers move by several hundred as you bend each finger.
Fail: a value stuck at 0 or 4095 means the 10kΩ resistor is missing or on the wrong leg.

Upload the complete firmware from Section 6. Open Serial Monitor at 115200.
Pass: status lines beginning with `#` during calibration, then continuous CSV lines like:
`243.18,11.90,-2.35,1874,1902,0,0`
Fail: if you only see `#` lines forever, the calibration never reached level 2 — keep moving the glove in a figure-8.

Close the Arduino Serial Monitor first — only one program can hold the port.

```
import glove_reader, time
glove_reader.start(port="COM3")     # your port here
time.sleep(3)
for _ in range(40):
    x, y, pen, gest, ready = glove_reader.get_state()
    fi, fm = glove_reader.get_flex()
    print(f"x {x:4d}  y {y:4d}  pen {'DOWN' if pen else 'up  '}"
          f"  gesture {int(gest)}  ready {int(ready)}"
          f"  flex {fi}/{fm}")
    time.sleep(0.1)
```

Pass: x and y change as you move your hand; pen shows DOWN when you bend your index finger.
Tune: if the cursor barely moves, raise `SCALE`. If it flies to the edge instantly, lower it.

```
python airwrite.py
```

Checklist:
1. Cursor dot appears on the canvas and follows your hand
2. Bending the index finger turns the dot red
3. Moving with the index bent leaves a white stroke
4. Bending the middle finger and holding triggers recognition
5. The recognised word appears on your laptop screen

10. Tuning Guide

Table 5: What to change when something feels wrong

Symptom	Setting	Change it to
Letters come out too small	`SCALE`	Raise from 12 to 18 or 25
Cursor flies off the canvas	`SCALE`	Lower from 12 to 6 or 8
Cursor drifts while holding still	`DEADZONE`	Raise from 0.35 to 0.6
Small movements ignored	`DEADZONE`	Lower from 0.35 to 0.2
Pen triggers too easily	`FLEX_INDEX_THRESHOLD`	Raise by 200–300
Pen will not trigger	`FLEX_INDEX_THRESHOLD`	Lower by 200–300
Strokes look jagged	`STROKE_WIDTH` in airwrite.py	Raise to 24–28
Writing is mirrored	In `_integrate`	Flip sign of `d_h`
Writing is upside down	In `_integrate`	Flip sign of `d_p`

11. Troubleshooting

Problem	Cause	Fix
`BNO055 NOT FOUND`	Wrong pin or 5V used	VIN to 3.3V, SDA 21, SCL 22
`Access is denied` on COM port	Serial Monitor still open	Close Arduino Serial Monitor
No COM port listed	USB driver missing	Install CP2102 or CH340 driver
Values freeze after a while	Loose wire at a knuckle	Add more slack at that joint
Flex stuck at 0 or 4095	Resistor missing / wrong leg	Rebuild the voltage divider
Cursor drifts on its own	Poor calibration	Redo the figure-8 for 30 seconds
Pen always down	Threshold too low	Raise `FLEX_INDEX_THRESHOLD`
Pen never down	Threshold too high	Lower `FLEX_INDEX_THRESHOLD`
Middle finger does nothing	Wrong threshold	Recalibrate `FLEX_MIDDLE_THRESHOLD`
Recognition always wrong	Strokes too small	Raise `SCALE`, write larger
ESP32 keeps resetting	Laptop USB underpowered	Use a rear USB port or powered hub